

Description Of Your Assignment:

Now that you have the software requirements that developers will need to implement the creation of the EventHub Ticketing application, you now need to move onto the next step: creating a vision of what this application will actually look like.

One of the most important things for you to understand is that when you determine what you think the application should look like, you will need to share your vision with other people. Words alone won't allow you to do this correctly. Instead, you need to create a diagram that shows the various components of the application and, most importantly, how you envision them working together.

Working With C4 Context Diagrams

Prior to getting to the deliverable part of this assignment, you could use online tools such as Structurizr Lite (https://structurizr.com/) to create C4 Context diagrams. This software was created by Simon Brown, who invented the C4 Model – it is an official C4 tool. It understands C4 concepts and generates Context. Preparing for your future, it will also generates Containers and generates Components. It also help you to keep diagrams synchronized. Please note that to use this tool, you will be required to write a small amount of DSL (text).

An example of DSL text would look like this:

```
workspace {

model {

user = person “Customer”

softwareSystem = softwareSystem “Banking System”

user -> softwareSystem “Uses”

}

views {

systemContext softwareSystem {

include *

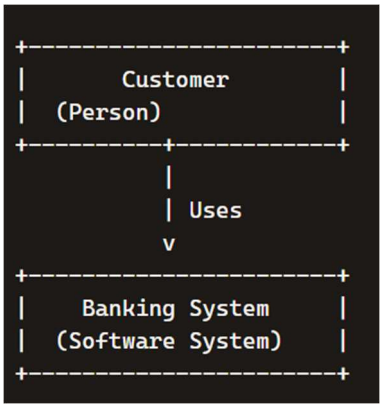
}

}

}
```

An example C4 Context diagram for this DSL text would look like this:





Example C4 context diagram

What You Need To Do:

Using the software requirements that you have created, you need to create C4 Context diagrams that show the top level design for the product. A key part of your design will be to show how each modules will interact with the other modules.

Change happens. You need to review the transcripts from call #2, based on what you learn from this discussion, you will need to update your software requirements.

Your task in this assignment is to use the updated software requirements from the last assignment and submit them to a GenAI tool and ask that it create two separate sets of C4 Context diagrams that could be used to create the EventHub Ticketing application along with pros and cons for each of the two models.

It is then your responsibility to take a look at the two possible solutions that have been presented to you and decide which one you think is the best one to proceed forward with. You then need to document why you made this decision.



End Result:

You will turn in the following items:

1. The two sets of C4 Context diagrams that were created – both the source file and the output file. [20% of your grade]
2. Your documentation on which C4 Context diagram you chose and why you chose it. [60% of your grade] *
3. The C4 Context diagram that you chose – both the source file and the output file. [20% of your grade]

Notes:

***** While you are allowed to use AI to work on building the context diagram, the question "am I allowed to use AI for documentation" needs an elaborate discussion. You can use AI to refine your reasoning or for understanding the trade-offs of both choices, but are the responsible owner of that reason. In other words, if asked later, (for example, during presentations) you need to be able to explain why you choose one architecture over the other.

Your documentation needs to reflect that.

Also, consider the following news article as you continue to work on the AI project:

"Shopify CEO says employees' 'slop grenades' are making more work for everyone else",
https://www.businessinsider.com/shopify-ceo-ai-slop-grenades-can-make-work-harder-2026-9

Good luck!



Call 2: Architecture of EventHub

Adaeze [Director]: Smaller group today. Thuy [QA] is at a testing conference and Darnell [USF/Customer] has a campus event. Priya [Lead Dev], welcome. You've read the requirements notes?

Priya [Lead Dev]: I have, plus the list of open questions. Before anyone draws a diagram, I want us to agree on the highest-level picture: what is the system, who touches it, and what sits outside it. Everything more detailed comes in the later design meeting. For the modeling itself my team will be using the C4 model, so by end of today we will know what to put in the context diagram, and the design calls for the container and component levels below it. I usually use Structurizr Lite, which builds the diagrams from a text file. As a result, the diagrams are in version control with the code and a change to a diagram gets reviewed the same way as any other change.

Marcus [Product Manager]: Fine by me. Let's start with the roles of the system.

Priya [Lead Dev]: From the requirements: one type of user for EventHub: a person who registers, browses, and buys. Students, parents, and fans in another country are all the same role from the system's point of view. Is that the complete list of people?

Marcus [Product Manager]: That's everyone. People buy tickets; the system does the rest.

Kenji [Security]: Agreed. One user role keeps the security work simple.

Priya [Lead Dev]: Then the top-level picture has a single human actor. Now the other direction: what external entities does EventHub depend on? External systems, outside services, anything the system talks to but we don't build.

Kenji [Security]: The exchange rates come from somewhere.

Marcus [Product Manager]: That's a configuration file we edit manually. The configuration file's inside the system; there's no rate provider on the internet we call.

Priya [Lead Dev]: We should settle that, because it changes the diagram. If rates are a file we ship with the application, the boundary contains them. If they're ever fetched from outside, that's an external system with an arrow. Which is it? And how do rates get updated: who edits them, and through what?

Marcus [Product Manager]: Manually, once a semester, I think. The mechanics were never discussed. Someone edits something.

Priya [Lead Dev]: "Someone edits something" is not enough to draw a diagram. I'm adding it to the open list; the design call should settle it.

Adaeze [Director]: Noted. What else crosses the boundary?

Priya [Lead Dev]: The payment step is pretend in version one, so no payment company sits outside the boundary. The confirmation question from Call 1 matters here too: if the answer ever becomes "send email," an email service appears outside the boundary with an arrow to it. As long as confirmation is on-screen only, nothing external is involved.



Marcus [Product Manager]: Adaeze [Director] settled that in the email thread: on-screen only, no email in version one. Draw it that way.

Priya [Lead Dev]: Good, then nothing external for confirmation. So the provisional context: one system, one human actor, and (pending the rates answer) possibly nothing external at all. A very simple diagram, which is fine.

Adaeze [Director]: Before you go inside, I have one constraint I want on the record. The application is written in Python, with Flask. It is cost effective, and my team members are familiar with that stack at this point, so this is not up for discussion.

Priya [Lead Dev]: Understood; it's a reasonable fit for a web application of this size. I have some more questions about the inside of the system that still affect the top level. First: one application or several? Flask works either way.

Kenji [Security]: For a first version with a pretend payment step, where realistic launch traffic is mostly one university plus early users?

Priya [Lead Dev]: Splitting into services buys independent scaling and deployment, at the cost of network calls and distributed failures, and it is much more to operate. A single application is simpler to build and to secure. I have my opinion, but the right answer depends on load and team size, and it deserves a written decision record either way.

Adaeze [Director]: What load are we designing for? The platform is open to anyone worldwide, but at launch the traffic will mostly be Darnell [USF/Customer]'s 26,000 students.

Marcus [Product Manager]: Right. A big ticket release might be a few hundred people at once in the first minutes, maybe more. Worldwide signups will come in slowly rather than all at once.

Priya [Lead Dev]: "A few hundred, maybe more." Can anyone put a number on it? Because a few hundred at once and ten thousand at once are different designs.

Marcus [Product Manager]: I'll try to get sales data from Darnell [USF/Customer]'s past events.

Priya [Lead Dev]: Please. There is one behavior at the top level that worries me more than any load number. Let's walk through a purchase: a buyer clicks buy. What must be true before the ticket count goes down?

Kenji [Security]: They're logged in. Quantity is valid. Event exists and isn't sold out. Currency is on the supported list.

Priya [Lead Dev]: And all of those checks plus the count going down need to happen as one indivisible step. Suppose two buyers want the last ticket and both pass the availability check at the same moment. Who gets it? If the answer is "both," we've sold a ticket that doesn't exist. I don't see this behavior specified anywhere.

Marcus [Product Manager]: What do other ticketing systems do?

Priya [Lead Dev]: Various things; the mechanism is a design-call topic. The *requirement*, "the system shall never sell more tickets than exist, even when purchases happen at the same time," is what's missing. Someone should write that sentence down this week rather than waiting for the design call.



Adaeze [Director]: Marcus [Product Manager], that's yours. Anything else at this level?

Kenji [Security]: One thing. Whatever the internal shape ends up being, the boundary rule I want stated now: every piece of data that crosses *into* the system (form fields, quantities, currency codes) is checked at the boundary. Nothing inside the system trusts data that came from outside it.

Priya [Lead Dev]: Agreed, and that rule is worth a sentence on the context diagram itself. Last question from me: is there any administrative or operational human in this picture? Someone who adds events, corrects a listing, investigates a complaint?

Marcus [Product Manager]: Events get loaded from a data file at setup for version one. Nobody needs a screen for that.

Kenji [Security]: And investigations go through the audit log, which someone reads. We haven't said who.

Priya [Lead Dev]: All right. Then the record for today: one actor, no confirmed external systems, Python and Flask as the given platform, single-application versus services deferred to a decision record, rates mechanics unresolved, and one new requirement sentence owed by Marcus [Product Manager]. I'll put the context diagram in the repository this week. The inside of the system is for the design call.

Adaeze [Director]: Good place to stop. Thanks all. Design call in two weeks, full attendance please.